

Redspher — Full Stack Developer

TypeScript · React · Node.js · AWS · REST ·
CI/CD · AI · LLM · MCP

Válasz-központú tudástár a technikai körre.

Minden tétel egy reálisan feltehető kérdés, plusz egy tömör, megvédhető válasz. A **kék vonallal jelölt** kérdések a stack alapján

legvalószínűbbek. A **borostyán jegyzet** a tipikus rákérdezést vagy csapdát jelöli, amit az interjúztató utána feszeget.

01 TypeScript

— KULCS

Az „erős TypeScript-tudás” kétszer is szerepel a hirdetésben — mélységre számíts, ne szintaktikai apróságokra.

Q. `interface` vs `type` — melyiket mikor választod?

Mindkettő objektum-alakot ír le. Az **interface** nyitott — támogatja a deklaráció-összeolvasztást (declaration merging) és az `extends`-et, ezért ez az alapértelmezés publikus/objektum-szerződésekhez és bármihez, amit egy fogyasztó kiegészíthet. A **type** zárt alias, de sokkal kifejezőbb: unió, metszet, tuple, mapped és conditional típusok, primitívek.

Ökölszabály: `interface` a bővíthető objektum-szerződésekhez; `type`, ha unióra vagy típusszintű számításra van szükség.

Rákérdezés: „Létezhet két azonos nevű interface?” Igen — összeolvadnak. Két azonos nevű `type` fordítási hiba. Pont ez az összeolvasztás az oka, hogy a könyvtárak interface-eket tesznek közzé.

Q. Magyarázd el a generikusokat egy valós megkötéssel. Mit ad a `<T extends ...>`?

A generikusok megőrzik a függvény polimorfizmusát, miközben megtartják a hívó pontos típusát ahelyett, hogy `any`-vé omlana össze. Egy megkötés leszűkíti, mi lehet a `T`, így biztonságosan hozzáférsz a tagjaihoz.

```
function prop<T, K extends keyof T>(  
  obj: T, key: K  
) : T[K] {  
  return obj[key];  
}
```

A `K extends keyof T` garantálja, hogy a kulcs létezik, és a visszatérési típus a pontos érték-típus lesz — egy elgépelte kulcs fordítási hiba.

Q. `unknown` VS `any` VS `never` ?

- **`any`** — teljesen kikapcsolja az ellenőrzést; ragadós és értelmetlenné teszi a típusozást. Kerülendő.
- **`unknown`** — top típus: bármit elfogad, de használat előtt kényszerít a szűkítésre. Ez a biztonságos `any` -helyettesítő a határokon (API-válaszok, `JSON.parse`).
- **`never`** — bottom típus: nincs értéke. Egy mindig dobó függvény visszatérési típusa, és az exhaustiveness-ellenőrzés eszköze `switch` -ben.

Q. Hogyan garantálsz, hogy egy unió feletti `switch` teljes (exhaustive)?

Rendeld a default ágat `never`-hez. Ha valaki hozzáad egy unió-tagot, de elfelejt egy case-t, az értékadás nem fordul le.

```
default: {  
  const _exhaustive: never = value;  
  throw new Error(`Kezeletlen: ${  
    _exhaustive`);  
}
```

Q. Mik a utility típusok, és melyiket használod ténylegesen?

Beépített mapped típusok, amelyek meglévő típusokat alakítanak át: `Partial<T>` (minden opcionális — patch payloadok), `Pick` / `Omit` (DTO-k származtatása egy modellből), `Required`, `Readonly`, `Record<K,V>`, valamint `ReturnType` / `Parameters` függvényekből való inferálásra. Az értéke: egyetlen igazságforrás — variánsokat származatsz duplikálás helyett.

Q. Type guard-ok és diszkriminált uniók?

A **diszkriminált unió** minden variánsnak közös literál mezőt ad (`kind: 'a' | 'b'`); ennek ellenőrzése leszűkíti az egész objektumot. A **felhasználói type guard** egy `x is Foo` típust visszaadó függvény, amely megtanít egy futásidejű ellenőrzést a fordítónak. Együtt a futásidejű validációt fordításidejű biztonsággá alakítják.

02 React

— KULCS

Hook-belsőiségekre, re-render okok magyarázatára és „miért lassú / hibás ez” jellegű szituációkra számíts.

Q. `useMemo` VS `useCallback` VS `React.memo` — melyik mit old meg, mikor felesleges?

- **`useMemo`** — egy *kiszámított értéket* memoizál rendereken át.
- **`useCallback`** — egy *függvény-identitást* memoizál; `useCallback(fn, d) ≡ useMemo(() => fn, d)`.
- **`React.memo`** — egy *komponenst* memoizál, kihagyva az újrenderelést, ha a propok sekélyen (shallow) egyenlők.

Csak együtt van értelmük: egy memoizált gyerek (`React.memo`) így is újrenderelődik, ha minden rendernél friss függvényt/objektumot adsz át neki — ezért kell a `useCallback` / `useMemo` a prop-identitás stabilizálására.

Csapda: Ne szórd mindenhová. A memoizációnak saját költsége van, és olcsó értékeknél nettó veszteség. Akkor nyúlj hozzá, ha drága a számítás, vagy propokat kell stabilizálnod memoizált gyerekekbe — ne alapból.

Q. Mi vált ki valójában egy re-renderet? Miért renderelődik újra, amikor „semmi sem változott”?

Egy komponens akkor renderelődik újra, ha: a **state**-je változik, a **szülője** újrarenderelődik, vagy egy **fogyasztott context** értéke változik. A „semmi sem változott” tipikus oka, hogy a szülő újrarenderelődik, és minden alkalommal **új objektum/tömb/függvény referenciát** ad át — referencia szerint nem egyenlő, még ha szerkezetileg azonos is.

Re-render $\neq$ DOM-frissítés. A React egyeztet (reconcile) a virtuális fát, és csak ott nyúl a DOM-hoz, ahol eltérés van; egy re-render nulla DOM-írást is eredményezhet.

Q. A Hookok szabályai — mik ezek, és *miért* léteznek?

Hookot csak **legfelső szinten** hívj (soha ne feltételben, ciklusban vagy beágyazott függvényben), és csak **React-függvényből**. A React a hook-állapotot *hívási sorrend* szerint tartja számon, nem név szerint — egy feltételes hook eltolja a sorrendet renderek között, és rossz állapot jön vissza.

Q. A `useEffect` függőségi tömbje — mi történik `[]`, `[dep]` és kihagyott esetben?

- **Kihagyott** — *minden* render után lefut.
- `[]` — egyszer, mount után fut (cleanup unmountkor).
- **[dep]** — mountkor és valahányszor a `dep` `Object.is` szerint változik.

A cleanup függvény a következő effekt előtt és unmountkor fut — így mondasz le feliratkozásokról, timerekről vagy folyamatban lévő fetchekről.

Csapda: Ha hazudsz a függőségekről a linter elnémitásához, elavult closure-ök keletkeznek — az effekt régi state-et/propot fog be. A valódi okot javítsd (functional update, `useRef`, vagy az érték behúzása), ne a tömböt vágd meg.

Q. Kontrollált vs kontrollálatlan komponensek?

Kontrollált: a React state az egyetlen igazságforrás — `value` + `onChange` .

Kiszámítható, könnyen validálható/transzformálható. **Kontrollálatlan:** a DOM tartja az értéket, `ref` -fel olvasod. Kevesebb kód, jó egyszerű vagy file inputokhoz. Alapból kontrolláltat használj mindenhez, amit gépelés közben validálsz vagy amire reagálsz.

Q. Kulcsok (key) listákban — miért ne a tömbindex?

A kulcsok segítik a Reactet elemeket instanciákhoz párosítani rendereken át. Az **index** kulcs átrendezésnél/beszúrásnál/törlésnél elromlik: a React rossz instanciát használ újra, így az input-state és a fókusz rossz sorhoz tapad. Használj stabil, egyedi id-t az adatból.

Q. Hogyan debuggolnál egy lassú listát / akadozó rendert?

- **Profiler** (React DevTools), hogy lásd, mely komponensek renderelnek és milyen gyakran.
- **Virtualizálás** hosszú listáknál, hogy csak a látható sorok mountolódjanak.
- Propok stabilizálása / komponensek szétbontása, hogy a re-render lokális maradjon.
- A ténylegesen drága számítások memoizálása.

Méréssel kezdj — sose optimalizálj vakon.

03 Node.js & Backend

— KULCS

A React app mögötti „backend szolgáltatásokat” fogszerűen építeni — az event loop és az async biztos téma.

Q. A Node egyszálú — hogyan kezel több ezer egyidejű kérést?

Egy szál futtatja a JavaScriptet, de az I/O **nem blokkoló**. Az **event loop** átadja az I/O-t (hálózat, lemez) az OS-nek vagy a libuv szálkészletének (thread pool), és továbbmegy; amikor az OS jelzi a befejezést, a callback sorba kerül és lefut. Így az egy szál sosem blokkol *várakozás* közben — csak akkor foglalt, amikor JS-t futtat. Ezért kiváló a Node az I/O-korlátos terheléshez, és ezért küzd a CPU-korlátossal (azt worker threadbe vagy külön szolgáltatásba told ki).

Q. Callback → Promise → async/await. Mit csinál valójában az `async/await`?

Ez szintaktikai cukor a Promise-ok felett. Az `await` szünetelteti az async függvényt, amíg a Promise le nem zárul, *anélkül, hogy blokkolná az event loopot* — közben más munka fut. Lineáris kóddá lapítja a „callback poklot”, és `try/catch`-et használhatsz async hibákhoz.

Rákérdezés: Szekvenciális vs párhuzamos `await`. Ciklusban `await`-elni sorbaállítja a független hívásokat; indítsd el őket együtt és `await Promise.all([...])`, ha nem függenek egymástól.

Q. Hogyan kezeled a hibákat async Express/Node kódban?

`try/catch` az `await`-elt hívások körül; továbbítás centralizált error middleware-hez `next(err)` -rel (vagy egy wrapperrel, ami elkapja a rejectionöket). Kulcsfontosságú: egy kezeletlen Promise rejection vagy egy csupasz callbackben dobott hiba leviheti a folyamatot — minden async ágnek kell `catch`, és egy legfelső szintű `unhandledRejection` handler a védőháló.

Q. Mi a middleware, és hogyan működik a lánc?

`(req, res, next)` szignatúrájú függvények, amelyek regisztrációs sorrendben futnak. Mindegyik vagy válaszol, vagy `next()` -tel továbbadja a vezérlést. Így rétegzed a keresztmetsző szempontokat — auth, logolás, body parsing, validáció, hibakezelés — anélkül, hogy a route-logikát szennyeznéd. A sorrend számít: auth a védett handler előtt, error middleware legutoljára.

**Q. `process.nextTick` VS `setImmediate`
VS `setTimeout(fn, 0)` ?**

A `process.nextTick` még azelőtt fut, hogy az event loop továbbmenne (legmagasabb prioritás — kiéhezteti a loopot, ha visszaélsz vele). A `setImmediate` a check fázisban fut, I/O után. A `setTimeout(fn, 0)` a timers fázisban fut, minimális késleltetéssel. A sorrend ismerete jelzi, hogy érted a loop fázisait.

04 REST API tervezés

*Kifejezetten a stackben van — tervezési
ítélőképességre számíts, nem bemagolt
státusz kódokra.*

Q. Mitől RESTful egy API? Tervezz egyet egy `orders` erőforrásra.

Erőforrás-orientált URL-ek (főnevek), HTTP igék a műveletekhez, állapotmentes kérések, helyes státusz kódok.

- `GET /orders` — lista · `GET /orders/:id` — egy elem
- `POST /orders` — létrehozás (→ `201` + `Location`)
- `PUT /orders/:id` — teljes csere · `PATCH` — részleges
- `DELETE /orders/:id` — törlés

Ige sosem megy az útvonalba (`/getOrders` hibás); a beágyazás kapcsolatokat mutat: `GET /orders/:id/items`.

Q. **PUT** vs **PATCH**, és mi az idempotencia?

A **PUT** lecseréli a teljes erőforrást; a **PATCH** részleges frissítést alkalmaz. **Idempotens** = ugyanaz a kérés megismételve ugyanabba a szerverállapotba vezet. A **GET**, **PUT**, **DELETE** idempotens; a **POST** nem (két POST két sort hoz létre). Ez a biztonságos kliens-újrapróbálkozáshoz számít gyenge hálózaton.

Q. Státuszkódok — amiket helyesen kell tudnod elhelyezni.

- **200** OK · **201** Created · **204** No Content
- **400** Bad Request · **401** nincs hitelesítve · **403** hitelesítve, de tiltott · **404** Not Found · **409** Conflict · **422** validáció sikertelen
- **500** szerverhiba · **503** nem elérhető

Klasszikus csapda: 401 vs 403 — a 401 = „nem tudom, ki vagy”, a 403 = „tudom, és így sem szabad”.

Q. Hogyan verziózol és lapozol egy API-t?

Verziózás: URL-prefix (`/v1/`) a leggyakoribb és cache-barát; a header-alapú tisztább, de kevésbé látható. **Lapozás:** az offset/limit egyszerű, de beszúrásoknál elcsúszik; a **kurzor-alapú** (keyset) stabil és jól skálázódik — nagy vagy élő adathalmazoknál ez az ajánlott.

05 AWS felhő

„AWS felhőkörnyezetben dolgozol” — szolgáltatás-választási érvelésre számíts. A Solutions Architect / Developer Associate certjeid ide tartoznak.

Q. Vezess végig az app AWS-en való hostolásán. Mely szolgáltatások, és miért?

- **Frontend (React):** statikus build **S3**-ban **CloudFront** mögött (CDN, TLS, cache).
- **Backend: Lambda + API Gateway** eseményvezérelt/csúcsos terheléshez, vagy **ECS/Fargate** hosszan futó konténerekhez.
- **Adat: RDS** (relációs) vagy **DynamoDB** (kulcs-érték, serverless skálázás).
- **Auth: Cognito. Titkok:** Secrets Manager / SSM. **Async:** SQS/SNS/EventBridge.

Kompromisszumként add elő — serverless (nincs üzemeltetés, cold start) vs konténerek (kontroll, stabilabb latencia).

Q. Lambda vs EC2 vs Fargate — hogyan választasz?

Lambda: eseményvezérelt, 15 percnél rövidebb feladatok, scale-to-zero, hívásonkénti fizetés — figyelj a cold startra és az időkorlátokra. **Fargate:** konténerek szerverkezelés nélkül — stabil szolgáltatások, nincs cold start. **EC2:** teljes kontroll / speciális igények, de tiéd a patchelés és skálázás. Döntési tengely: kéreisminta (csúcsos → Lambda, stabil → Fargate) és mennyi infra-kontroll kell.

Q. Mi az a cold start, és hogyan csökkented?

Az első hívás latenciája, amikor a Lambda új végrehajtási környezetet indít (runtime init + a kódod). Enyhítés: **provisioned concurrency**, kisebb bundle/kevesebb függőség, könnyebb runtime-ok. Azért releváns, mert ez egy serverless React+Node stack legfőbb csapdája.

Q. S3 + CloudFront egy React SPA-hoz — mi az az egy config, amit sokan elfelejtenek?

SPA kliensoldali routing: egy mély link, mint a `/orders/42`, nem talál S3 objektumot → 403/404. Javítás: az error válaszokat irányítsd vissza az `index.html`-re (200), hogy a React Router átvegye. Plusz: a hashelt asseteket cache-eld hosszan, de az `index.html`-t tartsd rövid cache-en, hogy a deployok átjussanak.

Q. IAM egy mondatban — role vs user vs policy?

A **policy** JSON jogosultsági dokumentum. A **user** hosszú életű identitás (személy). A **role** felvehető identitás ideiglenes hitelesítő adatokkal — ezt kellene a szolgáltatásoknak és Lambdáknak használniuk. Alapelv: **least privilege**, és role-ok a beégetett kulcsok helyett.

06 Git & CI/CD

Szerepel a stackben — gyakorlati workflow-kérdésekre számíts, nem parancs-trivián.

Q. `merge` vs `rebase` — mikor és miért?

A **merge** megőrzi a valós history-t egy merge committal — biztonságos megosztott branchekhez. A **rebase** újrátssza a commitokat egy új bázisra a lineáris history-ért — remek egy lokális feature branch rendezésére PR előtt. **Aranyszabály:** soha ne rebase-elj olyan branchet, amit mások már lehúztak — átírja a megosztott history-t.

Q. Mi történik valójában egy CI/CD pipeline-ban?

CI: minden push-nál — install, lint, type-check, teszt, build; fail fast, hogy hibás kód sose merge-elődjön. **CD:** main-re merge-nél — az artifact deployolása staging/prod-ra, gyakran automatikus rollbackkel. A lényeg: ismételhető, emberi hibától mentes út a committól a produkcióig.

Q. Hogyan tartod a titkokat és env configot Git-en kívül?

`.env` a `.gitignore`-ban; a valós értékeket a CI/CD titok-tárból vagy AWS Secrets Manager / SSM-ből injektálsz build- vagy futásidőben. Sose commitolj hitelesítő adatot — és ha egy kiszivárog, forgasd (rotate), ne csak a commitot töröld (a history megmarad).

07 AI · LLM

• MCP • — MEGKÜLÖNBÖZTETŐ

Agentek

*Erősen hangsúlyos: „AI-vezérelt megoldások, agent-alapú architektúrák (MCP, AI agentek, orkesztráció).”
Itt tudsz kitűnni. A szókincset tudd folyékonyan.*

Q. Mi az MCP (Model Context Protocol), és miért számít?

Egy nyílt szabvány az LLM-ek külső eszközökhöz, adatokhoz és rendszerekhez való csatlakoztatására egységes interfészen keresztül — gondolj rá úgy, mint „közös protokoll, hogy bármelyik modell bármelyik eszközzel beszélhessen”, integrációnkénti egyedi glue-kód helyett. Egy **MCP szerver** eszközöket/erőforrásokat tesz közzé; egy **MCP kliens** (a modell host-alkalmazása) felfedezi és meghívja őket. Szabványosítja azt, amit eddig mindenki kézzel drótozott össze function-callingként.

Miért érdekli őket: a hirdetés kifejezetten megnevezi az MCP-t. Már egy világos fogalmi válasz plusz „olvastam a specifikációt / kötöttem be szervert” is előrébb tesz azoknál, akik csak hallották a betűszót.

Q. Mi egy „AI agent” a sima LLM-híváshoz képest? Mit jelent az orkesztráció?

Egy sima **LLM-hívás** egy prompt be, egy completion ki. Egy **agent** a modellt egy hurokba csomagolja, amely képes **eszközöket használni, eredményeket megfigyelni és eldönteni a következő lépést** egy cél felé — tervezés → cselekvés → megfigyelés → ismétlés. Az **orkesztráció** ezeknek a lépéseknek (és gyakran több agentnek/eszköznek/ szolgáltatásnak) a megbízható koordinálása: útválasztás, sorrendezés, állapot, újrapróbálkozás, guardrail-ek.

Q. Mi az a RAG, és mikor használd?

Retrieval-Augmented Generation: releváns dokumentumok lekérése (általában embeddingek + vektor-tár segítségével) és beinjektálása a promptba, hogy a modell a *te* adataidból válaszoljon, ne csak a tanításából. Domain-tudáshoz, friss tényekhez és a hallucináció csökkentéséhez használd — a legtöbb üzleti esetben olcsóbb és gyorsabb, mint a finomhangolás.

Q. Function/tool calling — hogyan működik fejlesztői szemszögből?

Leírod az elérhető függvényeket (név, paraméterek, JSON séma) a modellnek; az egy strukturált hívással válaszol — melyik függvény, milyen argumentumokkal. **A te kódod futtatja le**, és visszaadja az eredményt, hogy a modell folytassa. A modell dönti el, *mit* hívjon; sosem futtat semmit maga. Az MCP lényegében ennek a mintának a szabványosított transzportja.

Q. Tokenek, context window, temperature — a gyakorlati gombok?

- **Token** — szórészlet-egység; a számlázás és a limitek tokenenként mennek.
- **Context window** — a max tokenek száma (prompt + kimenet), amit a modell egyszerre lát; túlcsonkítás → csonkolás vagy retrieval.
- **Temperature** — véletlenszerűség.
Alacsony = determinisztikus (kinyerés, osztályozás); magasabb = kreatív.

Q. Hogyan tartod megbízhatónak és biztonságosnak az LLM-funkciókat produkcióban?

Validáld/parse-old a modell kimenetét séma ellen (sose bízz a szabad szövegben), szűkíts tool callinggal, tegyél be guardrail-eket és timeoutokat, kezeld a nemdeterminisztikus hibamódokat, tarts human-in-the-loopot magas tétű műveleteknél, és logolj/értékelj. Ha az LLM-hívásokat *nem megbízható I/O-ként kezeled, amit validálnod kell*, pont azt a pragmatikus szemléletet mutatod, amit a hirdetés kér.

08 Üzleti → technikai

A hirdetés hangsúlyozza az üzleti igények skálázható megoldássá fordítását — egy „hogyan állnál neki X-nek” tervezési beszélgetésre számíts.

Q. „Fordíts egy homályos üzleti igényt technikai megoldássá” — hogyan állsz neki?

Előbb tisztázd a valódi kimenetet és a megkötéseket (kinek, milyen skála, latencia, must-have vs nice-to-have); mondd vissza. Aztán válaszd a **legegyszerűbb megoldást, ami a mai igényt kielégíti és nem zárja el a holnapit** — a kompromisszumokat mondd ki nyíltan ahelyett, hogy túlbonyolítanád. Mutasd meg, hogy üzleti értékre és karbantarthatóságra optimalizálsz, nem újdonságra.

Q. Hogyan teszel egy megoldást „skálázhatóvá” túltervezés nélkül?

Állapotmentesség (horizontális skálázás), lassú munka kitolása async sorokba, forró olvasások cache-elése, a valós lekérdezési minták indexelése, lapozás. De *mért* igényre skálázz — a korai elosztott bonyolultság önmagában is kudarc. „10×-re tervezz, ne 1000×-re, és mérd, mielőtt shardolsz.”

Q. A PHP legacy „plusz” — mit mondasz, ha rákérdeznek?

Légy őszinte a szintedről, majd keretezd konstruktívan: kényelmesen olvasol és karbantartol/integrálsz legacy kódot, és egy legacy PHP rendszerhez úgy állnál, hogy megérted a határait és tisztán integrálsz (pl. egy API-illesztésen keresztül) kockázatos újraírás helyett. Ők karbantartás/integrációként jelölték, nem zöldmezős fejlesztésként — ehhez a keretezéshez igazodj.

Q. Hová teszed a megosztott típusokat egy TS frontend és Node backend között?

Egy megosztott csomagba/modulba (monorepo workspace), amely exportálja a típusokat, amiket mindkét oldal importál — egyetlen igazságforrás az API-szerződésekhez, így egy backend-alakváltozás frontend fordítási hibaként jelenik meg. Bónusz: a típusok sémából való származtatása (pl. Zod), hogy a futásidejű validáció és a statikus típusok szinkronban maradjanak.